

第 15 章 系统运行与维护

信息系统在完成规划、分析、设计、开发、测试、交付后就给用户使用，系统便进入运行和维护阶段。系统运行和维护是一项长期工作，从大多数信息系统的实际情况看，系统运行与维护阶段占整个系统生命周期的比重为 60% ~ 80%。

在信息系统运行过程中，通常或多或少会出现在系统调试与测试阶段未发现的隐藏缺陷，抑或因用户提出新的特性需求所带来的系统功能的增强或扩展，即对系统实施改动，为此需要对系统进行科学的维护与管理，评价系统的工作质量和经济效益。

作为系统分析师，需要掌握系统运行与维护的原理、方法与技术，以更好地为信息系统的运行和维护提供有力的支撑。

系统运行和维护主要涉及系统的运行及故障处理、软件维护、系统实施及评价、遗留系统的处理、新旧系统的转换、系统的扩展等方面的内容。

15.1 运维技术指标

系统在完成开发并交付使用后系统就处于运行和维护阶段，在系统运行和维护过程中，难免会因为系统本身的异常，抑或外部环境等因素导致系统出现这样或那样的问题，从而引发系统业务的间断不可用或失效等。针对这些问题，需要制定必要的应对策略或措施以降低异常情况发生的概率，提升用户的服务体验。比如，可通过一些可量化的指标来对系统的运行和维护能力进行衡量，以便提升系统的运行和维护管理水平。

在系统运行和维护中，可使用平均故障修复时间（MTTR）、平均故障间隔时间（MTBF）、平均无故障时间（MTTF）和平均应答时间（Mean Time to Acknowledge, MTTA）等指标来帮助运维部门开展管理工作。

1. 平均故障修复时间

MTTR 是指系统可修复故障的平均维修时间，即在给定时间（时间段）内所发生的一次或多次系统故障的累计维修时间相对于故障次数的平均值。这里的系统故障维修时间是指从系统故障发生时刻起到系统被维修并恢复至正常状态所花费的时间，其中包括故障通知、故障排查诊断、故障系统修复所需测试时间等。

可以说，MTTR 对公司的利益有极大的影响。过长的 MTTR 意味着维修效率低下，有时会导致公司失去订单，或影响到与客户的关系。为缩短 MTTR，可采取提升运维团队能力的举措，如培训、故障处置演练等。另外，保持关键备件的备用量，做好有关异常情况应急预案的策划工作，都不失为有效的手段。

另外，从系统维护角度看，MTTR 中的 R 除了表示维修外，也可以表示恢复、响应或解决。尽管所有这些指标都和系统重新回到正常状态有关，但它们是从不同维度诠释系统的修复能力。

平均恢复时间是指设备或系统从故障中恢复所需的平均时间。这表示从系统因异常宕机到系统再次正常运行的整个过程持续的时间。这个指标表明系统异常恢复的速度。

平均响应时间是指从系统出现导致系统功能异常的故障告警开始系统到从故障中恢复所需的平均时间。这个指标通常用在网络安全中，用以衡量运维团队或系统消除系统攻击的效率。

平均解决时间是指系统出现故障后到完全解决系统故障为止所花费的平均时间，包括故障探测、问题诊断、研判，以及采取必要措施以避免此类故障再次发生所花费的时间。这个指标主要用于衡量发生意外事件并进行相应处置的反应能力。

由于 MTTR 是衡量运维团队实施故障修复进度的管理指标，因此运维团队应尽力做到 MTTR 尽可能小，可以通过提高运维团队修复效率来实现。实际工作中，MTTR 可以采用如下方式来估算，即：

$$\text{MTTR} = \text{给定时间段内消耗在系统修复上的总时间} / \text{维修次数}$$

例如，在一定时间周期内，系统出现了 6 次故障，并且将系统修复至完整功能所花费的总的维护时间为 2h（120min），那么，MTTR 估算为 $120/6 = 20\text{min/次}$ 。可理解为平均每次维修时间为 20min，即运维团队的故障平均恢复时间为 20 min。

2. 平均故障间隔时间

MTBF 是指系统两次故障之间的平均运行时间，即在给定周期内，系统发生一次或以上的故障之间累计运行时间相对故障次数的平均值。这里，每段系统运行时间从系统开始正常运行或故障恢复后开始计算，到下次故障发生为止。MTBF 用于衡量系统的可靠性。因此 MTBF 值越大，系统越可靠。

MTBF 指标可作为运维团队研判何时进行系统升级或对硬件进行维护的参考。如在对系统进行预防性维护后 MTBF 有所提高，则表明系统的可靠性得到了提高。在某种意义上，MTBF 的增大也间接说明了维护能力的提升。

由上可知，MTBF 是从一次故障到下一次故障之间经过的时间。理论上可按照如下方式计算：

$$\text{MTBF} = \text{多次故障之间系统总运行时间} / \text{故障总数}$$

例如，在一定周期内，系统累计运行了 1000h，期间发生了 2 次故障。因此，MTBF 为 $1000/2 = 500\text{h/次}$ 。此值表示系统平均每运行 500h 发生 1 次故障。跟踪并提升此指标值，则间接反映了系统可靠性的提升。

需要说明的是，MTBF 用于表示系统可靠性，它仅反映系统意外发生导致业务中断的情况，不包括计划内维护所导致的系统退出服务时间。可通过改善预防性维护过程，进行故障根源剖析并制定有效避免方案，或进行例行健康检查等手段来提升 MTBF。

3. 平均无故障时间

MTTF 是指可更换或不可修理系统或部件提供正常功能的平均持续时间，即指多个同类系统或部件从开始正常运行起到出现故障为止所持续时间相对同种系统或部件数量的平均值。MTTF 用于衡量不可修复系统或部件的可靠性，表示系统在失效之前可以运行的时间。MTTF

常用于衡量可更换或不可修理硬件寿命的。硬件 MTTF 的历史数据可作为运维人员掌握硬件预期使用寿命,有计划进行系统检查、维护工作的参考。

MTTF 是系统可靠性的主要指标,MTTF 较短,意味着系统频繁宕机、业务中断。MTTF 可采用下述方式计算:

$$\text{MTTF} = \text{给定时间周期内系统可正常运行总时间} / \text{故障总数}$$

例如,系统持续运行 100h,期间发生 5 次故障。此情况下,MTTF 为 $100/5=20\text{h}$ /次。这意味着需要平均每 20h 进行一次业务中断或更换硬件部件等。

4. 平均应答时间

MTTA 是运维团队响应业务部门的投诉、业务中断或异常事件所花费的平均时间。MTTA 体现了告警管理系统的效率,以及运维团队的响应能力。运维团队通过优化流程,尽可能将 MTTA 减至最短,可提升客户满意度,同时也降低运维成本。

MTTA 是监视运维团队响应性的措施之一。如果运维团队对系统异常事件分析处置时间过长,说明运维团队事件研判效率不高。

MTTA 可通过以下方式计算:

$$\text{MTTA} = \text{给定时间周期内系统出现告警到告警确认之间累计的总时间} / \text{事件总数}$$

例如,某个系统发生了 5 次告警事件,这些事件从告警触发开始到确认之间所花费时间分别为 6min、7min、9min、5min、8min。那么 MTTA 为 $(6+7+9+5+8)/5=7\text{min}$ /次。这意味着运维团队平均每 7min 应答 1 次。为进一步提升用户满意度,运维团队应优化异常事件处置流程,降低 MTTA。

综上所述,通过统计平均故障间隔时间 (MTBF),可反映出系统运行的可靠程度;通过平均故障修复时间 (MTTR) 来度量系统的修复能力;依据平均无故障时间 (MTTF) 可确定系统或硬件的业务持续性或寿命;利用平均应答时间 (MTTA),可跟踪运维团队的事件跟踪反应能力。

15.2 系统运行管理

系统运行管理的目的是对信息系统运行进行管控,记录其运行状态,进行必要的修改与扩展,使得信息系统更好地为管理和决策提供支持。其主要内容包括日常运行管理、系统运行情况记录、对系统运行情况的检查与评价等。完成这些工作是一个琐碎而细致的原始资料积累过程,不能忽视。

15.2.1 系统用户管理

系统用户管理是指管理用户的身份和权限,使用户在授权范围内对系统进行操作,防止非授权访问。用户管理的功能包括用户账号管理、权限管理、企业外部用户管理和用户安全审计。大量的统计数据表明,安全问题往往是从企业内部出现的,特别是用户身份的盗用,往往会造成一些重要数据的泄露或损坏。

1. 统一用户管理

在信息系统中，通常对用户进行统一的管理。例如，对于一个内部用户而言，身份识别管理的时间跨度从员工加入企业开始直到离开为止。进入企业后，新员工的资料就被添加到人力资源管理系统，系统会自动生成各种密码和授权，开始记录这名员工在企业里所做的任何访问活动。当员工离开时，网管只需将其从人力资源管理系统中删除，身份识别管理系统就会自动地将与该员工相关的授权全部删除。因此，进行统一的用户管理可以带来如下好处：

（1）用户使用更加方便。采用统一认证系统后，用户只需要使用同一个用户名和密码就可以登录所有允许他登录的系统，就能对各个应用系统进行访问。

（2）安全控制力度得到加强。管理人员可以集中控制用户的访问范围和权限，并对用户的行为进行审计。

（3）减轻管理人员的负担，提高工作效率。管理人员可以通过一个统一的管理界面集中管理用户信息，提高工作效率，减少人为失误的可能性。

2. 身份认证的方法

目前，信息系统中的用户身份认证方式主要有以下4种：

（1）用户名/密码方式。这种方式是最简单也是最常用的身份认证方法。但是，其安全性不太好，密码容易被猜测，或者被驻留在计算机内存中的木马程序或网络中的监听设备截获。

（2）集成电路（Integrated Circuit, IC）卡认证。IC卡是一种内置集成电路的芯片，芯片中存有与用户身份相关的数据。IC卡由专门的厂商通过专门的设备生产，理论上是不可复制的硬件，通过IC卡硬件不可复制性来保证用户身份不会被仿冒。但是，由于每次从IC卡中读取的数据是静态的，通过内存扫描或网络监听等技术，还是很容易截取到用户的身份验证信息，因此，IC卡还是存在安全隐患。

（3）动态密码。动态密码技术是一种让用户密码按照时间或使用次数不断变化、每个密码只能使用一次的技术。它采用一种叫做动态令牌的专用硬件，内置电源、密码生成芯片和显示屏，密码生成芯片运行专门的密码算法，根据当前时间或使用次数生成当前密码并显示在显示屏上。认证服务器也采用相同的算法计算当前的有效密码。二者进行对比验证，从而实现用户身份认证。它采用“一次一密码”的方法，即使黑客截获了一次密码，也无法利用这个密码来仿冒合法用户的身份，从而保证了用户身份的安全性。但是，如果客户端硬件与服务器端程序的时间或次数不能保持良好的同步，就有可能发生合法用户无法登录的问题。

（4）USB Key 认证。它采用软硬件相结合、“一次一密码”的强双因子认证模式，很好地解决了安全性与易用性之间的矛盾。USB Key 是一种 USB 接口的硬件设备，它内置单片机或智能卡芯片，可以存储用户的密码或数字证书，利用其内置的密码算法实现对用户身份的认证。

3. 用户安全审计

用户安全审计的主要功能是收集、保护和分析用户安全审计数据，形成用户安全审计报告。常见的用户安全审计报告包括如下内容：

- (1) 系统运行过程中通常会发生什么情况, 用户通常要登录访问哪些资源, 用户访问系统的时间分布特征等。利用这些基本信息发现和审核异常情况, 从而发现问题之所在。
- (2) 用户登录系统的时段记录, 审核个别用户在不寻常时间登录的情况。
- (3) 用户登录失败的记录, 审核某个账户在一段时间内多次登录失败的情况。

15.2.2 网络资源管理

网络资源是企业重要的 IT 基础设施, 包括局站、机房、管道、光缆、电缆、传输设备、光/电路、交换、数据、动力、接入、时钟同步、基站等。随着信息化建设的深入, 企业网络资源的规模不断扩大, 网络资源的管理和维护任务也越来越困难, 管理和维护工作的要求也越来越高。

1. 网络资源管理的功能

网络资源管理就是通过某种方式对网络资源进行调整, 使网络能正常、高效地运行, 可以把网络资源管理理解为广义的网络管理, 而网络管理主要包括性能管理、故障管理、配置管理、计费管理和安全管理 5 项功能, 这些功能提供了一个网络系统正常运行的基本保证。

(1) 性能管理。在使用最少的网络资源和具有最小延迟的前提下, 确保网络能提供可靠、连接的通信能力, 并使网络资源的使用达到最优化的程度。性能管理有监测和控制两大功能, 监测功能实现对网络中的活动进行跟踪, 控制功能实施相应调整来提高网络性能。

(2) 故障管理。检测、定位和排除网络硬件和软件中的故障。当出现故障时, 该功能确认故障, 并记录故障, 找出故障的位置并尽可能地排除这些故障。

(3) 配置管理。掌握和控制网络的状态, 包括网络内各个设备的状态及其连接关系。配置管理的典型方法是, 用逻辑图来描述所有的网络设备及其逻辑关系, 并将网络的确切物理布局, 以适当的比例映射到这个逻辑图上。用精心设计的各种图标来表示各种网络对象, 而这些图标又往往涂上不同颜色表示相应设备的不同状态。

(4) 计费管理。记录各用户和应用程序对网络资源的使用情况。计费管理提供计算一个特定网络或网段的运行成本的手段。

(5) 安全管理。对网络资源及其重要信息访问的约束和控制, 包括验证网络用户的访问权限和优先级、检测和记录未授权用户企图进行的不应有的操作等。

2. 网络资源管理系统

资源管理系统的定位是管理网络资源数据和支持网络资源的设计与分配。资源管理可以对数据进行收集、分类、增删改、显示、存储和检索, 保证数据的完整性和一致性, 支撑其上的各类应用系统, 支撑网管系统开通电路等。

一般来说, 网络资源管理系统应该对所有网络资源提供基于地图方式的查询、统计分析、资料配置和维护等基础管理功能, 并提供网络资源的物理路由图和逻辑拓扑图管理; 提供对设备的各种展开图、端子图等关联管理; 提供网络资源的关联管理、资源调度和割接管理; 提供对光/电路的路由配置、光/电路由信息查询功能; 提供完善的系统数据维护、安全管理、数据备份与恢复、版本控制等方面的系统运行管理功能。

15.2.3 软件资源管理

软件资源就是指企业整个环境中运行的软件和文档。软件资源管理是指优化管理信息的收集，对企业所拥有的软件授权数量和安装地点进行管理。软件资源管理还包括软件分发管理，即通过网络把新软件分发到各个站点，并完成安装和配置工作。

在项目管理中，软件资源可复用的程度是衡量企业软件能力成熟度的一个重要指标。有关软件能力成熟度的详细知识见 7.4 节。

1. 软件构件管理

软件构件是系统的一个可独立配置且具有可复用价值的单元，它驻留在计算机中，一般采用构件库的形式来进行管理。软件构件的有效利用离不开软件平台的支持，该平台包括构件的运行支撑环境、构件开发和组装环境、构件管理环境、基于构件的开发方法和开发过程等。

2. 软件分发管理

软件分发管理的支持工具可以自动完成软件部署的全过程，包括软件打包、分发、安装和配置等，甚至在特定的环境下可以根据不同事件的触发实现软件部署的回滚操作。在相应的管理工具支持下，软件分发管理可以自动化或半自动化地实现下列软件分发任务：

(1) 软件部署。信息系统管理人员可将软件包部署至遍布网络系统的目标计算机，对它们执行封装、复制、定位、推荐和跟踪。

(2) 安全补丁分发。通过结合系统清单和软件分发，安全修补程序管理功能能够显示计算机需要的重要系统和安全升级，然后有效地分发这些升级。

(3) 远程管理和控制。远程诊断工具可以帮助技术支持人员及时、准确地获得关键的系统信息，能够花费较少的时间诊断故障，并以远程方式解决问题。

3. 文档管理

软件文档也称文件，通常指一些记录的数据和数据媒体，它具有固定不变的形式，可被人 and 计算机阅读。软件文档的编制在软件开发工作中占有突出的地位和相当大的工作量。在整个软件生存期中，各种文档作为半成品或最终产品，会不断地被生成、修改或补充。

15.3 系统故障管理

在信息系统运行过程中，由于人力、技术、资源、管理等方面的限制，故障是难免的，即使是电信、金融等信息化工作相对领先的行业，也难以保证自己的信息业务可以高枕无忧。因此，如何进行有效的故障管理是系统运行与维护过程中一项非常重要的工作。

故障管理的主要目标是尽可能快地恢复系统运行，尽量减少故障对业务运营的不利影响，以确保最好的服务质量和可用性。在故障管理中，影响度、紧迫性和优先级是描述故障的 3 个维度，它们相互联系又相互区别。故障管理包括故障监视、故障调查、故障排查、恢复处理和故障收尾 5 项基本活动。为了实现对故障流程的完善管理，需要对故障管理的整个流程进行跟

踪, 并做相应的处理记录。

15.3.1 故障监视

在信息系统运行过程中, 不可避免会出现一些由于系统自身问题, 或者是任何不符合标准的操作规程, 已经发生或者可能发生的系统运行中止和服务质量下降的事件, 这就是故障。故障监视是故障管理流程的第一项基础活动, 大多数故障都是在故障监视活动中发现的。

1. 设置待监视项目

系统故障有很多类型。不同的系统故障有不同的特征, 对系统和整个企业的业务影响程度可能不同, 处理解决的难易度也不同。在进行故障监视时要充分考虑故障的影响范围、紧迫性, 对影响较大的故障类别进行重点监视, 借助先进的自动化监视管理工具, 启动更多的系统监视功能, 或者投入更多的人力、物力和财力。这样, 在相关部门发现故障时, 才能尽快根据影响程度设置故障处理的优先级, 尽快进入管理流程。

故障接触人员在故障监视过程中起着重要作用。为了在监视过程中尽快发现和应对故障, 同时防止非规范操作扩大故障对系统和业务的影响, 需要对各类故障的接触人员进行严格管理。应该针对不同的故障接触人员制定监视职责和相关操作手册, 而故障接触人员应该严格按照规定执行操作和进行故障报告。同时, 故障接触人员本身及其活动也应当作为监视的对象。

2. 监视的内容和方法

根据故障管理的实践, 人员、操作规范性、系统硬件和软件是故障监视的重点内容。自然灾害因素由于难以预计和控制, 需要进行相关风险分析, 可采取容灾防范措施来应对。

对系统硬件设备的监视包括各主机服务器及其主要部件、专门的存储设备、网络交换机、路由器等, 主要采用管理监控工具, 它们通常具有自动监测、跟踪和报警的功能。

对软件的监视主要针对其应用性能、软件缺陷和变更需求。对软件的性能监控也可以借助一些管理监控工具; 由于应用系统主要面向用户, 软件缺陷通常由专门的测试工程师负责监视, 或者在使用过程中由用户方发现并提出; 变更需求是在用户使用和监视的过程中发现的。

需要监视的人员包括系统操作员、系统开发工程师、用户、来访者, 甚至包括系统所在机房的清洁工和运输公司的职工等。要对他们与系统接触过程中的行为进行跟踪和记录, 预防或及早发现不规范操作带来的系统故障。

15.3.2 故障调查

故障调查就是收集故障信息、确定故障位置和调查故障原因的过程。

1. 收集故障信息

故障信息的收集方式分为自动收集和人工收集。通常, 系统本身具有相应的故障信息收集功能, 可以通过专门的系统监控工具或系统日志等方式进行自动收集; 另外, 系统运行过程中出现的故障会直接反映在系统的用户一方, 或者由相关 IT 部门在执行系统检查和维护时发现,

这类故障信息的收集方式则属于人工收集。

系统发生故障后，如果有必要则需要进行故障隔离，保留故障现场信息，以利于故障信息的收集工作。

2. 确定故障位置

在硬件设备方面，故障定位比较简单，也比较常见。计算机系统都配备了较完善的诊断测试手段，提供详细的故障维修指南，对大部分故障可以实现准确定位。对外围设备的故障检测应采用脱机检测与联机检测两种方式，以检查是哪种设备的哪个部分出现了故障；对于网络设备的故障，可以通过专门的命令或工具来进行测试和定位。

在软件和数据方面，故障定位比较复杂，通常需要经过软件调试的过程，以准确定位错误的代码行。

3. 调查故障原因

故障原因调查一般是在故障经由初步支持没有得到解决时进行的。

当用户、服务台员工和其他 IT 部门人员发现某故障时，或者系统检测到某故障时，就将其报告给服务台，服务台将基本信息输入故障数据库并报告给故障管理人员。故障管理人员根据服务台提供的信息和故障数据库信息，判断此故障是否与已有故障相同或相似。如果相同或相似，则更新故障信息和建立原故障的从属记录，并在必要时修改原故障的影响度和优先级；否则，就创建新的故障记录。

故障管理将给每个故障一个唯一的编号，记录一些基本的故障分析信息（例如，时间、症状、位置、用户和受影响的服务和硬件等），并补充其他故障信息（例如，与用户的交互信息和配置数据等）。故障管理需要判断故障是否严重，如果严重，就先向管理层报告并将有关情况告知用户，再采取进一步行动；如果不严重，就直接进入故障原因调查和分析。

根据实践经验的总结和有关统计数据，导致信息系统故障的原因大致可以分为以下 7 类：

（1）按计划进行硬件、操作系统维护操作时引起的故障，例如，扩充硬盘和进行操作系统升级等。

（2）应用性故障，例如，应用软件的性能问题、缺陷和系统应用变更等。

（3）人为操作故障，例如，人员的误操作和不按规定的非标准操作引起的故障等。

（4）系统软件故障，例如，操作系统崩溃和数据库的各类故障等。

（5）系统硬件故障，例如，硬盘、网卡、路由器等设备的损坏。

（6）相关设备故障，例如，停电时 UPS 失效导致服务中断等。

（7）灾难和灾害，例如，火灾、地震、洪水和战争等。

15.3.3 故障排查和恢复处理

故障经过调查和记录，基本上能得到可以获取的故障信息，接下来就是排障的初步支持。其目的是能够尽可能快地恢复用户的正常工作，尽量避免或者减少故障对系统服务的影响。不能通过初步支持来解决的故障在经过故障调查和定位分析后进行故障信息更新，然后由技术支持小组根据更新后的故障信息、提议的权宜措施和解决方案以及有关的变更请求，来进一步解

决故障并恢复服务，同时更新有关故障信息。

1. 硬件设备故障的恢复

主机故障需要启用系统备份进行恢复，对于比较重要的系统，一般采用冗余的结构，如主备或双活，抑或以资源池方式提供保护。当系统出现故障时，就用一个备用的模块或另外处于同等功能的主机，或资源池里的其他主机来顶替它并重新运行。

当遇到线路故障或者网络连接问题时，需要利用备用电路或者改变通信路径等恢复方法。针对系统其他相关设备的故障，应分析、查找设备有关技术与非技术上的故障原因，如有需要应与供应商取得联系，进行设备维修、调换、更新后，使设备运行正常。

2. 数据库故障的恢复

数据库故障主要分为事务故障、系统故障和介质故障，不同故障的恢复方法也不同。当系统运行过程中发生故障，利用数据库后备副本和日志文件就可以将数据库恢复到故障前的某一一致性状态。

3. 应用软件故障的恢复

信息系统中的应用软件由于设计不当，或者程序代码有误，都会造成系统故障。对于软件类故障的处理方法，通常是通过软件调试，找出错误的代码，然后进行修改和测试。为了提高系统的可靠性，也可以采取软件容错技术。

15.3.4 故障收尾

解决故障和恢复服务后，就到了故障收尾阶段。该阶段主要是基于上一阶段更新后的故障记录和已解决的故障，与用户一起确认故障是否被成功解决，并更新故障信息和故障记录。

15.4 软件系统维护

系统维护是系统运行过程中的重要一环，是保证系统正常运行，或为了改正错误或满足新需求而修改系统的活动，包括软件维护（程序维护）、数据维护、代码维护、设备维护，以及机构和人员的变动等。在整个信息系统的维护过程中，软件维护是最重要的工作，也是最难的工作。因此，本节只讨论软件维护的问题。

15.4.1 软件维护概述

在系统交付使用后，改变系统的任何工作都可以被称为维护。与硬件不同，软件系统在构建时就包含了变化，软件并不会老化或需要周期性的维护。软件维护是指在软件交付使用之后，直至软件被淘汰的整个时期内，为了改正错误或满足新的需求而修改软件的活动。软件的维护活动基于“软件是可维护的”这一基本前提。

1. 软件维护性

根据国家标准《系统与软件工程 系统与软件质量要求和评价（SQuaRE）第10部分：系统

与软件质量模型》（GB/T 25000.10—2016），软件维护性是指产品或系统能够被预期的维护人员修改的有效性和效率的程度。修改包括纠正、改进或软件对环境、需求和功能规格说明变化的适应。修改还包括那些由专业支持人员实施的，以及那些由业务或操作人员、最终用户实施的。维护性包括安装更新和安装升级。维护性可以被解释为便于维护活动的一种产品或系统固有能力，或者为了产品或系统维护的目标维护人员所经历的使用质量。

GB/T 25000.10—2016 标准还规定了维护性的 6 个子特性：

（1）模块化。指由多个独立组件组成的系统或计算机软件，其中一个或多个组件的变更对其他组件产生的影响最小的程度。

（2）可重用性。软件能够被用于多个系统或其他软件构建的程度。

（3）易分析性。可以评估预期变更（变更产品或系统的一个或多个部分）对产品或系统的影响、诊断产品的缺陷或失效原因、识别待修改部分的有效性和效率的程度。

（4）易修改性。产品或系统可以被有效地、有效率地修改，且不会引入缺陷或降低现有产品质量的程度。（注：修改包括编码、设计、文档和验证的变更。模块化和易分析性会影响易修改性。）

（5）易测试性。能够为产品、系统或组件建立测试准则，并通过测试执行来确定测试准则是否被满足的有效性和效率的程度。

（6）维护性的依从性。产品或系统遵循与维护性相关的标准、约定或法规以及类似规定的程度。

2. 可维护性度量

人们一直期望对软件的可维护性做出定量度量，但要做到这一点并不容易。国家标准《系统与软件工程 系统与软件质量要求与评价（SQuaRE）第 23 部分：系统与软件产品质量测量》（GB/T 25000.23—2019）具体规定了如何从软件的外部 and 内部两个方面来度量可维护性，但是，这些规定比较烦琐，实用性比较差。因此，下面从实用角度来介绍可维护性的度量方法。

在软件外部，可以用 MTTR 来度量软件的可维护性，它指出处理一个有错误的软件需要花费的平均时间。为此，需要详细记录分析问题需要的时间、确定改动方案的时间、执行改动花费的时间、测试改动花费的时间，以及其他管理花费的时间。

在软件内部，可以通过度量软件的复杂性来间接度量可维护性。与软件复杂性相关的因素有环路数、软件规模等。

（1）环路数。环路数是 McCabe 在 1976 年提出的，因此也称为 McCabe 环路，它基于一个程序模块的程序图中环路的个数来度量软件复杂性。环路数可以反映源代码结构的复杂度，通过观察环路数的增长幅度，可以比较多种维护方案的优劣，选择对环路数影响最小的方案作为最优方案。

（2）软件规模。通常可以认为软件包含的构件越多，软件就越复杂，可维护性就越差。

（3）其他因素。其他因素包括嵌套深度、系统用户数等。

对于软件内部的可维护性，迄今还没有一个典型的、全面的、通用的模型，需要结合

不同开发团队自身的经验,建立合适的经验模型。但毋庸置疑,软件良好的设计方法无疑对软件的可维护性有着积极的作用。比如模块化设计中模块划分的合理性,如模块的可分解性(Decomposability)、可组合性(Composability)、可理解性(Understandability)、可持续性(Continuity)和保护性(Protection)等,以及模块的高内聚和低耦合性等,在设计方面应审慎考量,并加以应用,以便为软件系统的后期维护提供帮助。

3. 软件维护的分类

在系统运行过程中,软件需要维护的原因是多样的。根据维护的原因不同,可以将软件维护分为以下4类:

(1) 改正性维护。为了识别和纠正软件错误、改正软件性能上的缺陷、排除实施中的误使用,应当进行的诊断和改正错误的过程称为改正性维护。

(2) 适应性维护。在使用过程中,外部环境(新的硬、软件配置)、数据环境(数据库、数据格式、数据输入/输出方式、数据存储介质)可能发生变化。为使软件适应这种变化,而去修改软件的过程称为适应性维护。

(3) 完善性维护。在软件的使用过程中,用户往往会对软件提出新的功能与性能要求,为了满足这些要求,需要修改或再开发软件,以扩充软件功能、增强软件性能、改进加工效率、提高软件的可维护性。这种情况下进行的维护活动称为完善性维护。

(4) 预防性维护。预防性维护是指预先提高软件的可维护性、可靠性等,为以后进一步改进软件打下良好基础。通常,预防性维护可定义为“把今天的方法学用于昨天的系统以满足明天的需要”。也就是说,采用先进的软件工程方法对需要维护的软件或软件中的某一部分(重新)进行设计、编码和测试。

15.4.2 软件维护的影响因素

软件维护阶段占整个软件生命周期60%以上的时间,因此,分析影响软件维护的因素,提高软件可维护性就显得十分重要。软件维护的影响因素很多,主要有以下几个方面:

(1) 业务因素。因为系统在线运行,某些系统要保证7×24h运行,维护人员必须寻找一种途径,在不影响用户业务的情况下实现改动。

(2) 理解的局限性。统计数据显示,50%左右的软件维护工作量花在理解要修改的软件上。有时用户的理解也会出现问题,约有超过半数的维护问题源自用户技能或理解的缺乏。同时,维护人员也要具备一些人际技巧,努力理解不同用户的思维方式,以说服的方式处理掉一些问题。

(3) 对待维护的优先级问题。有时开发商会倾向于维持现有系统的运行,而客户更迫切地需要新功能,甚至一个新系统。

(4) 维护人员的积极性。通常,维护人员被认为是第二阶层,程序员大都认为设计和开发比维护工作更具技巧性和挑战性。

(5) 测试的困难。测试人员很难预测设计或代码改动带来的影响,使得测试很难做到充分。另外,有些系统只能在测试环境或备份系统中进行测试,上线后则不允许测试,由于无法精确地再现真实环境,也使得测试有一定的局限性。

为尽可能地降低这些因素的影响，维护人员经常要在长期和短期目标之间进行权衡，决定什么时候牺牲质量来换取速度。除此之外，还可以从以下方面来提高软件的可维护性。

1. 采用软件工程方法

软件危机从某种意义上可以看成是软件维护的危机，由此诞生的软件工程也可以看成是提高软件可维护性的工程。软件工程的采用，使得软件开发过程进一步规范，强制性地产生了一系列的文档，这些文档从各个阶段、各个方面对软件的结构、原理加以说明，极大地丰富了维护所需的资源，增加了软件的可维护性。所以，对于维护人员来说，文档比程序源码更为重要。

2. 注重可维护性的开发过程

要在开发过程中提高软件的可维护性，必须从如何使软件易于分析、易于测试、易于修改出发进行考虑。具体如下：

（1）在需求分析阶段，应该对将来要改进的和可能会修改的部分加以明确；对软件的跨平台可移植性进行讨论，形成解决方案。

（2）在设计阶段，应该尽量遵循“高内聚，低耦合”的设计原则，对将来可能要修改的地方，采用灵活的易于扩充的设计方案；考虑跨平台可移植性的设计；加大可复用构件的设计力度。

（3）在编码阶段，应该采用科学的代码规范，强化注释的力度，保证注释的质量，这一点对于将来的维护非常重要；加大可复用构件的使用力度。

（4）在测试阶段，一方面，测试本质上是减少了各种维护的工作量，尤其是改正性维护，也就是说，测试做好了，以后的维护量就少了；另一方面，测试相关的文档（包括测试计划、测试用例、测试报告等）是维护后的回归测试的基础，如果测试阶段的文档不全，维护后几乎无法进行回归测试，维护的质量也就无法保证。

（5）在维护阶段，要有严格的配置管理，每一次维护工作之后，都要按照配置关联，同步更新维护有关的系统文档和用户文档，保证系统的一致性。同时，在大的维护之后，交付之前，一定要及时做好用户的培训工作。否则，就会使用户因为工作受挫折而产生不满。在实际工作中，有些所谓的“错误”可能只是用户使用手册描述不清楚造成的。

15.4.3 软件维护管理

不管是哪种类型的维护，都需要类似开发的过程。从本质上说，维护过程是修改和压缩了的开发过程，但维护的范围更广，需要跟踪和控制的东西更多。维护的重点是保持对系统日常功能的控制和对系统改动的控制，完善现有可接受的功能，防止系统性能下降到不可接受的程度。

1. 软件维护组织

维护工作是需要相当大的投入的，关键行业的企业大都有自己的维护队伍。有实力的软件企业应该建立专业的维护部门，除维护自有产品外，也可承接维护外包业务。对于大多数的中小软件企业，虽然没有专门的维护组织，但非正式地明确责任却是非常必要的，需要指定维护负责人，组建临时的维护小组，明确维护流程及各人的职责，可以专设维护配置员，也可由维

护负责人兼任。

如果同时有多个系统需要维护,可再设维护管理员,监督协调各维护小组。对于大型软件系统的维护,设置维护管理员是非常必要的,由他协调和同步各维护小组的工作。维护配置员则对维护活动需要的资源以及维护过程中产生的各种文档进行标识和配置。由于维护几乎必然会带来变动,因此,需要维护配置员至少做好变动控制,包括变动的时间、执行变动的人员、变动的内容、变动的结果、批准变动的人员、变动的通知范围和变动的级别等。

2. 软件维护工作流程

软件维护工作流程如图 15-1 所示。

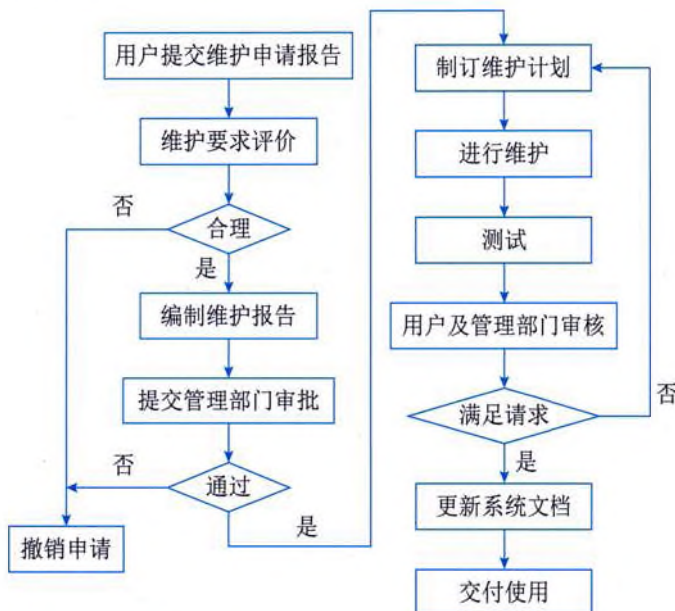


图 15-1 软件维护流程图

用户首先以书面形式的维护申请报告向维护管理员提出维护申请。对于改正性维护,报告中必须完整描述出现错误的环境,包括输入/输出数据和其他系统状态信息;对于适应性维护和完善性维护,应在报告中提出简要的需求规格说明书。

维护管理员根据用户提交的申请,召集相关的系统管理员对维护申请报告的内容进行审核和评价。如果情况属实,则编制维护报告,并将其提交给维护管理部门审批。维护管理部门基于整个系统,从合理性和技术可行性两个方面对维护要求进行分析和审查,并对修改所产生的影响做充分的估计。对于不妥的维护要求,在与用户协商后,予以修改或撤销。

通过审批的维护报告,维护管理员要根据具体情况制订维护计划。对于改正性维护,先估计其缓急程度,如果维护要求十分紧急,严重影响系统的运行,则应立即安排并修改系统;如果问题不是很严重,则可与其他维护项目结合起来,从维护开发资源上统筹安排;对于适应性维护或完善性维护,高优先级的将被安排在维护计划中,低优先级的可作为一个新的项目来组织开发。

维护管理员将维护计划下达给系统管理员。在真正执行维护工作之前，系统管理员要根据单位的实际情况制订一个维护实施计划，计划维护工作中的具体实施步骤与细节。然后，由系统管理员按实施计划进行具体的维护和修改工作。系统维护时，要对数据采取妥善的保护措施。远程维护时，应事先通知相关人员。修改后应经过严格测试，以验证维护工作的质量。测试通过后，再由用户和管理部门对其进行审核确认，不能完全满足要求的应返工进行修改。只有经确认的维护成果才能对系统的相应文档进行更新，最后交付用户使用。

需要说明的是，并非所有的维护活动都要完全按照上述流程进行。例如，如果需要维护的软件系统支撑着客户的核心业务，必须7×24h运行，这样的系统一旦发生恶性软件问题（例如，数据库崩溃等）时，维护组织就会以“救火队”的方式迅速展开维护，没有时间进行计划评审，此时，软件开发商应派出技术过硬的维护小组，以最大限度地降低风险和损失。

3. 保存维护记录

在维护活动中需要及时记录维护的有关信息，用于考察维护技术的有效性，估计软件的“优良”程度，确定维护的实际代价。同时，这些记录将作为后续评价活动的依据。

维护记录的主要内容包括源代码行数、使用的开发语言、程序的安装日期、从安装至今程序运行的次数、从安装至今程序失效的次数、程序变动的标识、因变动而增加的源代码行数、因变动而删除的源代码行数、每个改动耗费的人·时数、程序改动的日期、修改者的姓名、维护需求表的标识、新维护需求是否源于以前的维护工作、维护类型、维护开始和完成日期、累计用于维护的人·时数、已完成的维护创造的直接和间接的效益等。

对于每项维护工作，都要收集上述数据，可以基于这些数据建立起维护数据库。

4. 评价维护活动

如果缺乏有效的数据就无法评价维护活动，因此，需要在维护阶段保存维护记录，借助维护记录就可以对维护工作做一些定量的统计。可以从以下几个方面进行度量：

- (1) 每次程序运行平均失效的次数。
- (2) 用于每一类维护活动的总人·时数。
- (3) 平均每个程序、每种开发语言和每种维护类型所做的程序变动数。
- (4) 维护活动中增加或删除一条源代码花费的平均人·时数。
- (5) 以前维护工作引起的新维护占总维护需求的比例。
- (6) 维护需求表的平均周转时间。
- (7) 不同维护类型所占的百分比。

度量结果可以作为以后调整维护工作的参考，对于合理规划维护工作量、优化资源分配、针对性地强化对参与某类维护的人员的技术培训等方面可以起到积极作用。

15.5 系统评价

企业信息化是有风险的，信息系统规模越大，功能越复杂，风险也就越大。目前，在国内的信息化建设项目中，绝大多数用户（业主）无法组织队伍对信息系统建设进行专业化管理，

难以胜任从可行性分析、规划设计、招标、方案评审到工程验收全过程的管理与组织协调工作,建设方和承建方在信息系统建设过程中存在严重的信息不对称问题。要解决这个问题,就需要借助第三方的力量来对信息系统工程项目进行监控和管理,对信息系统进行审计和评价。

15.5.1 信息系统建设

信息系统工程的承建方(IT企业)一般在技术方面有着巨大的优势,并且拥有与众多客户打交道的能力与经验,对用户心理已经做了很好的分析。他们往往不顾自身实力而大胆向用户承诺自己根本不可能完成的目标,或者为了自己的利益拼命推销用户根本就不需要的功能。于是用户很容易被IT供应商所描绘的美好前景打动,很快就做出了决策。在信息系统建设过程中,建设方由于其自身技术力量的限制,只能任由承建方的“摆布”。另一方面,也有一些建设方对信息系统的需求本来就是模糊的,或者无法清楚地表达自己的需求,在工程实施过程中或工程完成之后一再追加需求,而不愿追加相应的费用,引起供求双方纠纷。

针对以上问题,除了需要用户提高自身水平,厂商提高服务质量以外,引进监理机制,借助第三方的技术和经验来规范项目的实施,保障项目的进度和质量就非常必要。

按照信息系统工程监理相关规定,对信息系统工程进行监理时,需要依法设立且具备相应资质的信息系统工程监理单位,其受业主单位委托,并依据国家有关法律法规、技术标准和信息系统工程监理合同,对信息系统工程项目实施监督管理。

15.5.2 信息系统评价

系统评价是对系统运行一段时间后的技术性能和经济效益等方面的评价,是对信息系统审计工作的延伸。评价的目的是检查系统是否达到了预期的目标,技术性能是否达到了设计的要求,系统的各种资源是否得到充分利用,经济效益是否理想,并指出系统的长处与不足,为以后系统的改进和扩展提供依据。

1. 评价流程

对于信息系统的评价,首先应该确定系统评价人员、评价对象、评价目标、评价指标和评价原则及策略等,制订相应的评价计划。无论是内部评价,还是外部评价,都要遵循以下工作程序:

(1) 确定评价对象,下达评价通知书,组织成立评价工作组和专家咨询组。评价通知书应载明评价任务、评价目的、评价依据、评价人员、评价时间和有关要求等事项。

(2) 拟定评价工作方案,收集基础资料。工作方案是评价工作组进行评估活动的工作安排,其主要内容包括评价对象、评价目的、评价依据、评价项目负责人、评价工作人员及工作时间安排,拟用评价方法、选用评价标准、准备评价资料和有关工作要求等。

(3) 对评价对象实施评价,征求专家意见和将意见反馈给企业,撰写评价报告。评价工作组依据企业报送的资料进行基本评价。如果是企业内部进行评价,可以只对部分内容进行评价,根据企业委托的职责而定;如果是外部评价,则要有委托部门的指令,按照计划行事。

(4) 评价工作组将评价报告报送专家咨询组复核,向委托人送达评价报告和选择公布评价结果,建立评价项目档案。

评价工作正式开始前,评价工作组可以按照评价基本要求,组织企业有关人员进行自测。

企业自测带有自愿和预备性质，自测报告应该有完整的工作底稿备查。评价工作组取得的评价结论应该与企业自测结论进行对照，及时对评价结论进行补充和修改，对评价工作组的评价结论和企业自测结论相差较大的，要进行基础资料的核对，找出差异的原因。

2. 评价指标

信息系统的评价是一项难度很大的工作，它涉及多目标评价问题。目前，大部分的系统评价还处于非结构化的阶段，只能对部分评价内容列出可度量指标，其他的还只能利用定性的方法来考量。总体而言，可以从系统性能、效益、建设等方面进行评价。

- 系统性能评价。信息系统的性能是指系统的各个组成部分，包括计算机硬件、软件、人员和各种规章制度，有机地结合在一起，作为一个整体对使用者所表现出来的特性。系统性能的好坏直接影响到系统的运行与维护，决定了运行和应用的长期效果，决定了系统的生命力。因此，系统性能评价是信息系统评价的主要内容。性能评价的指标体系一般包括可靠性、系统效率、可维护性、可扩充性、可移植性、实用性、适应性和安全保密性等。
- 系统效益评价。系统效益评价是指对系统的经济效益和社会效益等做出评价。经济效益评价又称为直接效益评价，社会效益评价又称为间接效益评价。由于社会效益评价难度较大，一般以经济效益评价为主。与经济效益有关的指标包括系统投资额、系统运行维护费用、运行信息系统而带来的收益和投资回收期等。
- 系统建设评价。系统建设评价分配在信息系统生命周期的各个阶段的阶段评审之中。在信息系统生命周期的不同阶段，系统评价的作用是不同的。例如，在系统规划阶段，重点关注如何识别满足业务目标的信息系统；在系统实现阶段，在于理解IT战略后，识别、开发或获取、实施信息系统解决方案，保持项目的方向。

1) 评价指标设计

应根据效能评价的目标，构建和分解评价指标维度。评价指标设计应遵循但不限于下述方式：

(1) 效率驱动方式：基于信息系统支撑的业务分类角度设计评价指标。当评价重点关注信息系统对各类业务支撑效果与价值的情况时，宜采用效率驱动方式。

(2) 能力驱动方式：基于支撑信息系统正常运行运转的能力（如组织管理能力、人力资金保障能力等）是否成熟、完善角度设计评价指标。当评价重点关注信息系统可以正常运转运行时，宜采用能力驱动方式。

(3) 效果驱动方式：基于信息系统运行产出物（如效率提升、成本降低等）角度设计评价指标。当评价目标重点关注信息系统运行整体效果时，宜采用效果驱动方式。

(4) 效益驱动方式：基于信息系统用户对象体验结果角度设计评价指标。当评价目标重点关注用户对信息系统运转运行的认可度、满意度时，宜采用效益驱动方式。

2) 评价指标类型设计

应在确定效能评价内容的基础上，结合指标数据的采集、来源渠道、覆盖范围和使用模式

等,明确评价指标的操作、用途等类型属性。常用的评价指标类型有:

- (1) 静态指标与动态指标。
- (2) 定量指标与定性指标。
- (3) 原生指标与衍生指标。
- (4) 相关性指标与因果性指标。
- (5) 先见性指标与后见性指标。

3) 评价指标权重设计

评价指标权重设计使用的方法主要有主观赋权法、客观赋权法和组合赋权法。

(1) 主观赋权法。当评价逻辑关系、重要性程度须体现指标制定者的评估意图、侧重点等主观考虑时,宜采用主观赋权法。主观赋权法主要包括:

- 专家调查法:通过问卷填写、调研访谈、头脑风暴等方式,由专家依据知识和经验,对评价指标重要程度进行判断、评估。
- 层次分析法:依据各评价指标的相互关联影响以及隶属关系,明确评价指标在指标体系中的层级及重要性,形成多层次的评价模型,从最底层指标相对于上级以至最高层指标的重要性来确定权重。
- 环比评分法:通过依次确定各评价指标的重要性系数来确定权重。
- 因素成对比较法:通过评价指标成对比较,依据比较结果所反映的重要性来确定权重。

(2) 客观赋权法。当评价指标拥有足够多的样本数据,能够通过数据公式反映逻辑关系、重要性程度时,宜采用客观赋权法。客观赋权法主要包括:

- 主成分分析法:通过正交变换将一组可能存在相关性的变量转换为一组线性不相关的变量来确定权重。
- 熵值法:通过计算评价指标的离散程度,并依据离散程度的大小来确定评价指标权重。
- 变异系数法:通过变异系数来比较评价指标之间的重要性,借此确定权重。
- 标准离差法:通过计算评价指标值的标准差,根据变异程度大小所对应的重要性程度来确定权重。
- CRITIC法:通过计算不同指标权重方案中,单一评价指标的表现差异性以及指标间冲突性来确定权重。

(3) 组合赋权法。当以上两种情形均存在时,宜使用组合赋权法。组合赋权法,即组合使用主观赋权法和客观赋权法,它是基于指标数据之间的内在规律和主观经验对评价指标确定权重。

3. 系统改进建议

系统改进建议是系统评价的最后一个环节,它是评价的最终结果。应根据实际数据,结合事先制定的指标,给出相应的、合理的评价建议。

在信息系统评价工作完成后,评价小组应根据相关评价指标,收集相应评价数据,然后根据这些指标和实际数据,检测开发过程中的缺陷,形成一个总的评价报告。评价报告是由评价工作组完成全部评价工作后,对信息系统的运行状况、效益状况和价值贡献等进行对比分析、

客观判断，形成的综合结论的文本文件。

评价报告由正文和附录两部分组成。正文的主要内容包括所评价的信息系统的基本情况描述、主要各项指标对比分析、评价结论、评价依据和评价方法等；附录包括有关评价工作的基础文件和数据资料。评价报告要维护被评价企业的商业秘密，必须客观、公正、准确地描述信息系统的实际状况和后续发展能力，并提出改进建议。

15.6 遗留系统处置

遗留系统（Legacy System）是指任何基本上不能进行修改和演化以满足新的变化了的业务需求的信息系统，它通常具有以下特点：

（1）系统虽然完成企业中许多重要的业务管理工作，但仍然不能完全满足要求。一般实现业务处理电子化及部分企业管理功能，很少涉及经营决策。

（2）系统在性能上已经落后，采用的技术已经过时。例如，多采用主机/终端形式或小型机系统，软件使用汇编语言或第三代程序设计语言的早期版本开发，使用文件系统而不是数据库。

（3）通常是大型的软件系统，已经融入企业的业务运作和决策管理机制之中，维护工作十分困难。

（4）没有使用现代信息系统建设方法进行管理和开发，现在基本上已经没有文档，很难理解。

在企业信息系统升级改造过程中，如何处理和利用遗留系统，成为新系统建设的重要组成部分。处理恰当与否，直接关系到新系统的成败和开发效率。

针对遗留系统的处理，需要进行全方位考量。首先要对它进行剖析，看它都有哪些利用价值，或哪些限制，抑或对业务的影响，以及响应程度大小等。然后根据分析结果，确定对它的处理策略，比如可以采取继续维护、某种形式的重构或替代策略，或者多策略并用。总的来说，遗留系统的处理，往往是多种策略的折中和权衡。

可见，在遗留系统处置上，对遗留系统的整体评价，以及演化方式的选择是极为重要的环节。

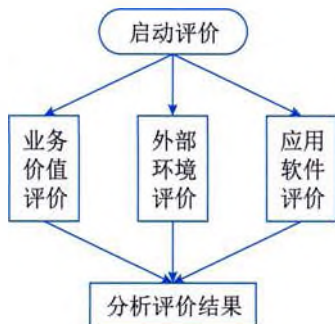


图 15-2 评价活动

15.6.1 遗留系统的评价

对遗留系统评价的目的是获得对遗留系统全面的理解，这是遗留系统演化的基础和起点。评价主要涉及对遗留系统的技术水准、商业价值和与之关联的企业特征等方面进行考量，结果作为对遗留系统选择处理策略的依据。评价方法由一系列活动组成，如图 15-2 所示。

1. 启动评价

评价是为了获得对遗留系统的足够深度的理解，从技术、

业务和企业角度对系统的理解为系统处理策略提供了基础。开始评价前,需要了解以下问题:

(1) 对企业来说,遗留系统是至关重要的。在评价过程中,可能会发现系统对企业的继续运作产生的影响不大。在这种情况下,就没有必要考虑系统的演化问题。

(2) 企业目标是什么。从战略观点来看,系统分析师必须理解企业目标,因为信息系统建设的目的是把企业战略目标转化为信息系统的目标,企业目标产生演化需求。

(3) 演化需求是什么。演化需求来自企业目标和评价活动。需求必须是可见的,以便决定现有系统是否能满足需求。

(4) 所期望的系统生命周期是多长。系统的生命周期由软件和服务的能力决定,一旦系统硬件或支撑软件过时,系统的有效性就会受到限制。

(5) 系统使用期限是多久。如果系统的使用期限只是短期的,就没有必要花费成本来演化系统。相反,如果系统将在相当长的时期内支持主要业务流程,则必须进行演化。

(6) 系统的技术状态如何。例如,如果应用软件的技术状况很差,则很难理解,维护成本会很高。

(7) 企业是否愿意改变。企业对改变的态度是遗留系统演化成功的关键因素之一。

(8) 企业是否有能力承受演化。企业的技术成熟度、员工的素质和支撑工具的先进性等都是影响演化的因素。

2. 业务价值评价

业务价值评价的目标是判断遗留系统对企业的重要性。在多数情况下,重要业务流程的改变意味着遗留系统现在仅仅具有外围价值,修改这种系统只需花费少许财力和物力。在其他情况下,系统的业务价值很大,需要继续运行与维护。

可以在概要和详细两个级别上进行遗留系统的业务价值评价。概要评价为更加详细的分析提供信息,包括向有关专家进行咨询、问卷调查,在问卷的基础上进行分析和评价;详细评价包括遗留系统不符合业务规范的风险分析,这种分析十分费时,最好由业务分析师来完成详细级的评价工作。

3. 外部环境评价

系统的外部技术环境是指硬件、支撑软件和企业 IT 基础设施的统一体。

(1) 硬件。硬件评价可以采用概要级和详细级。概要评价把遗留系统作为一个整体,提供硬件质量估算;详细评价包括识别系统中的每个部件。在这两种情况下,必须识别一系列特征,用作评价的基础。特征的选择取决于要评价的系统,系统的一些常见特征有供应商、维护成本、失效率、使用年限、功能和性能等。

(2) 支撑软件。系统的支撑软件环境由许多部分组成,可包括操作系统、数据库、事务处理程序、编译器、网络软件和应用软件等。一般来说,支撑软件是依赖于某个硬件的,应用软件依赖于系统软件。在评价过程中,必须考虑这种依赖性。

(3) 企业 IT 基础设施。企业 IT 基础设施包括开发和维护系统的企业职责和运行该系统的企业职责(两者可能为同一个企业),这些基础设施难以评价,但对遗留系统的演化却起到关键作用。在评价中必须考虑以下问题:企业和使用者的类型、开发组织的技术成熟度、企业的培

训过程、系统支持人员的技术水平、企业是否愿意改变等。

4. 应用软件评价

应用软件评价可以分为系统级和部件级。系统级评价把整个系统看作不可分割的“原子”，评价时不考虑系统的任何部分；部件级评价关注系统的每个子系统，考虑每个子系统的特征，包括复杂性、数据、文档、外部依赖性、合法性、维护记录、大小和安全性等指标。

5. 分析评价结果

评价活动将产生硬件、支撑软件、企业 IT 基础设施和应用软件的特征值矩阵。这些特征值体现了遗留系统当前的技术因素，其加权平均值代表了系统的技术水平，计算公式如下：

$$OR = (P_1ORH + P_2ORS + P_3OAF + P_4ORA) / 4$$

其中，ORH 是硬件的评价值，ORS 是支撑软件的评价值，OAF 是企业 IT 基础设施的评价值，ORA 是应用软件的评价值， $P_i (1 \leq i \leq 4)$ 分别是它们的加权系数，即第 i 个评价值对遗留系统的影响因子。把对技术水平的全面评价结果与业务评价进行比较，可以为系统演化提供第一手的资料。具体方法是按照业务评价分和技术水平分值的情况，把评价结果分为 4 种类型，如图 15-3 所示。

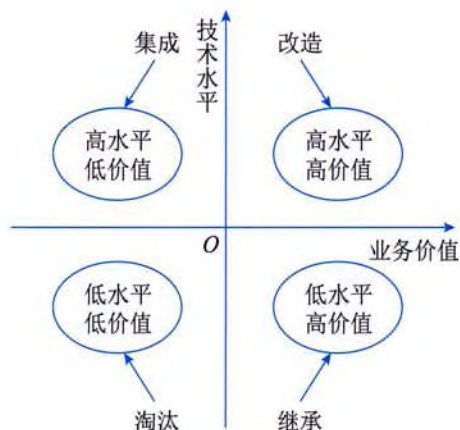


图 15-3 评价结果分析

15.6.2 遗留系统的演化

在图 15-3 中，把对遗留系统的评价结果分列在坐标的 4 个象限内。对处在不同象限的遗留系统采取不同策略加以演化。

1. 淘汰策略

第三象限为低水平、低价值区，即遗留系统的技术含量较低，且具有较低的业务价值。对这种遗留系统的演化策略为淘汰，即全面重新开发新的系统以代替遗留系统。完全淘汰是一种极端性策略，一般是企业的业务产生了根本变化，遗留系统已经基本上不再适应企业运作的需要；或者是遗留系统的维护人员、维护文档资料都丢失了。经过评价，发现将遗留系统完全淘汰，开发全新的系统比改造旧系统从成本上考虑更合算。

对遗留系统的完全淘汰是企业资源的根本浪费,系统分析师应该善于“变废为宝”,通过对遗留系统功能的理解和借鉴,可以帮助设计新系统,降低新系统开发的风险。

2. 继承策略

第四象限为低水平、高价值区,即遗留系统的技术含量较低,已经不能满足企业运作的功能或性能要求,但具有较高的商业价值,目前企业的业务尚紧密依赖该系统。这种遗留系统的演化策略称为继承。在开发新系统时,需要完全兼容遗留系统的功能模型和数据模型。为了保证业务的连续性,新老系统必须并行运行一段时间,再逐渐切换到新系统上运行。

3. 改造策略

第一象限为高水平、高价值区,即遗留系统的技术含量较高,本身还有极大的生命力。系统具有较高的业务价值,基本上能够满足企业业务运作和决策支持的需要。这种系统可能建成的时间还很短,这种遗留系统的演化策略称为改造。

改造包括系统功能的增强和数据模型的改造两个方面。系统功能的增强是指在原有系统的基础上增加新的应用要求,对遗留系统本身不做改变;数据模型的改造是指将遗留系统的旧的数据模型向新的数据模型的转化。

4. 集成策略

第二象限为高水平、低价值区,即遗留系统的技术含量较高,但其业务价值较低,可能只完成某个部门(或子公司)的业务管理。这种系统在各自的局部领域里工作良好,但对于整个企业来说,存在多个这样的系统,不同的系统基于不同的平台、不同的数据模型,形成了一个信息孤岛。对这种遗留系统的演化策略称为集成。有关企业应用集成的知识见6.8节。

要注意的是,本节所介绍的遗留系统演化策略具有通用性。在实际工程项目中,遇到处理遗留系统的问题时,要具体情况具体分析,选择最佳的演化策略,既要保护用户的已有投资,又要保证系统能满足用户当前(甚至未来)的需求,且具有一定的先进性。

15.7 新旧系统转换

当新系统开发完毕投入运行,要取代现有系统时,就要进行系统转换。系统转换是指运用某种方式,由现有系统的工作方式向新系统工作方式的转换过程,也是系统设备、数据、人员等的转换过程。在系统转换时,必须协调新旧系统之间的关系,否则将造成紊乱与中断,从而导致一定的经济损失。在系统转换过程中,需要考虑多个方面的问题,包括成本、风险、应急措施和人员的培训等。

在系统转换与交接之前,需要做好一些准备工作,包括数据准备、系统文档准备、人员培训和设备安装,以及系统试运行。系统试运行是指在系统没有正式转换之前所进行的试验运行,它是系统调试工作的延续。系统试运行时要输入各种真实数据,记录系统运行状况和产生的数据,比较现有系统与新系统输出的结果,同时对新系统的操作方式进行考查,测试系统运行、响应速度等性能指标。

15.7.1 新旧系统的转换策略

系统转换可分为两种情况，一种是现有信息系统被功能更强大的新系统所取代，例如，网络作业系统取代了单机系统，开放性系统取代了封闭性系统，标准化的系统取代了不规范的系统，更安全的系统取代了不太安全的系统，当然，最常见的还是系统功能与性能的提高；另一种是计算机信息系统取代了原有的手工作业系统。

不管是哪一种情况，系统转换的工作量都比较大。系统转换本质上也是一个工程项目，也需要进行可行性分析和需求分析，需要制订项目管理计划。6.3.3 小节介绍的项目开发计划的各种要素同样适用于系统转换计划的制订。此外，现有系统从启用到被新系统取代，在其使用期间往往积累了大量珍贵的历史数据，其中许多历史数据都是新系统顺利启用所必需的。因此，系统转换计划应该包括从现有系统向新系统的数据迁移计划。有关数据转换和迁移的知识将在 15.7.2 小节中介绍。

在实施新旧系统转换时，转换的策略通常有三种，如图 15-4 所示。

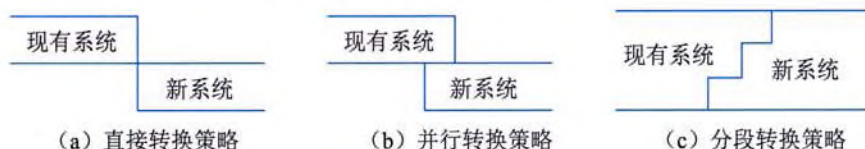


图 15-4 系统转换的方式

1. 直接转换策略

直接转换就是在原有系统停止运行的某一时刻，新系统立即投入运行，中间没有过渡阶段。直接转换的示意图如图 15-4 (a) 所示。采用这种方式时，人力和费用最省，适用于新系统不太复杂或现有系统完全不能使用的场合，但是，新系统在转换之前必须经过详细而严格的测试，转换时应做好准备，万一新系统不能达到预期目的时，必须采取相应措施。

直接转换的新系统是完全重构的系统，可能采用了全新的技术平台和软件来构建，或者用户业务和使用方式发生了剧烈变化，对原有系统只能进行淘汰处理。采用这种策略的优点是新系统能够非常灵活地适应业务需要，功能齐全、结构合理、系统稳定、扩展性强，整个信息系统的利用率比较高。采用这种策略也存在着一些问题，列举如下：

(1) 新旧系统之间的转换代价比较大。

(2) 由于需要一套比较完整的业务需求，开发新系统的周期比较长，一次性投资巨大，未经广泛使用并证明是成熟可靠的新技术平台通常具有一定的技术风险。

(3) 旧系统通常积累下了大量的业务数据，必须将业务数据的录入、转换、检查以及在新系统中的重建作为重要的工作进行考虑，尽量减小在新旧系统转换的时候对用户现有业务的冲击。

(4) 需要考虑诸如维持新系统运行的日常开销，由于使用习惯改变带来的学习时间、培训人员的成本等因素。

2. 并行转换策略

并行转换就是新系统和现有系统并行工作一段时间，经过这段时间的试运行后，再用新系

统正式替换现有系统。在并行工作期间,手工处理和计算机处理系统并存,一旦新系统有问题就可以暂时停止而不会影响现有系统的正常工作。并行转换过程如图15-4(b)所示。

在并行转换的实施过程中,首先以现有系统的作业为正式作业,新系统的处理结果作为校核之用,经过一段时间的运行,在验证新系统处理准确可靠后,现有系统退出运行。根据系统的复杂程度和规模大小不同,并行运行的时间一般可在两三个月到一年之间。

采用并行转换的风险较小,在转换期间还可同时比较新旧两个系统的性能,并让系统操作员和其他有关人员得到全面培训。因此,对于一些较大的信息系统,或处理过程复杂、数据重要的系统,并行转换是一种最常用的转换方式。但是,由于在并行运行期间,要两套班子或两种处理方式同时并存,人力和费用消耗较大,转换的周期长,并且难以控制新旧系统中的数据变化。这就要求做好转换计划并加强管理,在新旧系统验证吻合后要及时停止现有系统的运行。

3. 分段转换策略

分段转换策略也称为逐步转换策略,这种转换方式是直接转换方式和并行转换方式的结合,采取分期分批逐步转换,如图15-4(c)所示。一般比较大的系统采用这种转换方式较为适宜,它能保证平稳运行,费用也不太高;或者现有系统比较稳定,能够适应自身业务发展需要,或新旧系统转换风险很大(例如,在线订票系统、银行的中间业务系统等),也可以采用分段转换策略。

采用分段转换时,各子系统的转换次序及转换的具体步骤,均应根据具体情况灵活考虑。通常可采用如下策略:

(1) 按功能分阶段逐步转换。首先确定新系统中的一个主要的业务功能率先投入使用,在该功能运行正常后再逐步增加其他功能。

(2) 按部门分阶段逐步转换。先选择系统中的一个合适的部门,在该部门运行新系统,获得成功后再逐步扩大到其他部门。这个首先运行新系统的部门可以是业务量较少的,这样比较安全可靠;也可以是业务最繁忙的,这样见效大,但风险也大。

(3) 按机器设备分阶段逐步转换。先从简单的设备开始转换,再推广到整个系统。例如,对于联机系统,可先用单机进行批处理,然后用终端实现联机系统。对于分布式系统,可以先用两台计算机联网,以后再逐步扩大范围,最终实现分布式系统。

分段转换策略的优点是,新旧系统的转换震动比较小,用户容易接受。但由于是采用渐进方式,导致新旧系统的转换周期过长,同时由于需求的变化,给新系统的稳定造成比较大的影响。而且,分段转换策略对系统的设计和实现都有一定的要求,在转换过程中,需要开发新旧系统之间的接口,还需要制订阶段性的转换目标和计划。

15.7.2 数据转换和迁移

数据转换和迁移是新旧系统转换交接的主要工作之一。为使数据能平滑迁移到新系统中,在新系统设计阶段就要尽量保留现有系统中合理的数据结构,这样才能尽可能降低数据迁移的工作量和转换难度。但是,由于新系统的引入,数据迁移工作是个必然的过程,现有系统中的

数据可以通过定制开发的转换工具软件翻译为新系统可以接受的数据格式。

数据转换和迁移工作的原则是数据不丢失。许多无法自动转换的数据，必要时通过手工方式补录进入新系统。数据迁移对系统切换乃至新系统的运行有着十分重要的意义。数据迁移的质量是新系统成功上线的重要前提，同时也是新系统今后稳定运行的有力保障。如果数据迁移失败，新系统将不能正常启用；如果数据迁移的质量较差，没能屏蔽全部的垃圾数据，对新系统将会造成很大的隐患，新系统一旦访问这些垃圾数据，可能会由这些垃圾数据产生新的错误数据，严重时还会导致系统异常。相反，成功的数据迁移可以有效地保障新系统的顺利运行，而且能够继承珍贵的历史数据。

1. 数据迁移的方法

系统转换时的数据迁移不同于从 OLTP 到数据仓库的数据抽取。后者主要将 OLTP 系统在上次抽取后所发生的数据变化同步到数据仓库，这种同步在每个抽取周期都要进行，一般以天为单位。而数据迁移是将需要的历史数据一次或分几次转换到新系统，其最主要的特点是需要短时间内完成大批量数据的抽取、清洗和装载。

数据迁移的主要方法大致有三种，分别是系统切换前通过工具迁移、系统切换前采用手工录入和系统切换后通过新系统生成。

(1) 系统切换前通过工具迁移。在系统切换前，利用 ETL 工具把现有系统中的历史数据抽取、转换，并装载到新系统中去。这种方法是数据迁移最主要，也是最快捷的方法。其实施的前提是，历史数据可用并且能够映射到新系统中。这种迁移方式既可一次实现，也可以分次实现。一次迁移的优点是迁移实施的过程短，相对分次迁移，迁移时涉及的问题少，风险相对比较低。其缺点是工作强度比较大，由于实施迁移的人员需要一直监控迁移的过程，如果迁移所需的时间比较长，工作人员会很疲劳。一次迁移的前提是新旧系统数据库差异不大，在允许的宕机时间内可以完成所有数据量的迁移；分次迁移可以将任务分开，有效地解决了数据量大和宕机时间短之间的矛盾。但是分次切换导致数据多次合并，增加了出错的概率，同时为了保持整体数据的一致性，分次迁移时需要对先切换的数据进行同步，增加了迁移的复杂度。

(2) 系统切换前采用手工录入。在系统切换前，组织相关人员把需要的数据手工录入到新系统中。这种方法消耗的人力、物力比较大，同时出错率也比较高。这种方法主要针对新旧系统数据结构存在特定差异的情况，即对于新系统启用时必需的期初数据，无法从现有的历史数据中得到。对于这部分期初数据，就可以在系统切换前通过手工录入。

(3) 系统切换后通过新系统生成。在系统切换后，通过新系统的相关功能，或为此专门开发的配套程序生成所需要的数据。通常根据已经迁移到新系统中的原始数据来生成所需要的结果数据。这种方法可以减少迁移的数据量。

2. 数据迁移前的准备工作

数据迁移的实施可以分为三个阶段，分别是数据迁移前的准备、数据转换与迁移和数据迁移后的校验。由于数据迁移的特点，大量的工作都需要在准备阶段完成，充分而周到的准备工作是完成数据迁移的主要基础。具体而言，要做好以下 7 个方面的工作：

- (1) 待迁移数据源的详细说明, 包括数据的存放方式、数据量和数据的时间跨度。
- (2) 建立新旧系统数据库的数据字典, 对现有系统的历史数据进行质量分析, 以及新旧系统数据结构的差异分析。
- (3) 新旧系统代码数据的差异分析。
- (4) 建立新旧系统数据库表的映射关系, 以及对无法映射字段的处理方法。
- (5) 开发或购买、部署 ETL 工具。
- (6) 编写数据转换的测试计划和校验程序。
- (7) 制定数据转换的应急措施。

3. 数据转换与迁移

在数据转换与迁移阶段, 首先需要制定数据转换的详细实施步骤和流程, 准备数据迁移环境。然后要做好业务上的准备, 结束未处理完的业务事项, 将其告一段落。接下来使数据转换和迁移涉及的技术都得到充分测试, 最后实施数据转换和迁移。

数据转换与迁移程序大致可以分为抽取、转换与装载三个过程。数据抽取、转换是根据新旧系统数据库的映射关系进行的, 转换步骤一般还要包含数据清洗的过程。数据清洗主要是针对源数据库中, 对出现二义性、重复、不完整、违反业务或逻辑规则等问题的数据进行相应的清洗操作。在清洗之前需要进行数据质量分析, 以找出存在问题的数据。数据装载是通过装载工具或自行编写的 SQL 程序将抽取、转换后的结果数据加载到目标数据库中。

数据抽取前, 需要做大量的准备工作, 具体如下:

- (1) 针对目标数据库中的每张数据表, 根据映射关系中记录的转换加工描述, 建立抽取函数。该映射关系是准备阶段进行数据差异分析的结果。
- (2) 根据抽取函数的 SQL 语句进行优化。通常可以采用的优化方式有调整相关参数的设置、启动并行查询、采用优化器、创建临时表、对源数据表做分析和增加索引等。
- (3) 建立调度控制表, 包括 ETL 函数定义表 (记录抽取函数、转换函数、清洗函数和装载函数的名称和参数)、抽取调度表 (记录待调度的抽取函数)、装载调度表 (记录待调度的装载信息)、抽取日志表 (记录各个抽取函数调度的起始时间和结束时间, 以及抽取的正确或错误信息) 和装载日志表 (记录各个装载过程调度的起始时间和结束时间, 以及装载过程执行的正确或错误信息)。
- (4) 建立调度控制程序, 该调度控制程序根据抽取调度表动态调度抽取函数, 并将抽取的数据保存到文件中。

数据转换的工作在 ETL 过程中主要体现为对源数据的清洗和代码数据的转换。数据清洗主要用于清洗源数据中的垃圾数据, 可以分为抽取前清洗、抽取中清洗和抽取后清洗。ETL 对源数据主要采用抽取前清洗。对代码表的转换可以考虑在抽取前转换和在抽取过程中进行转换。具体转换操作如下:

- (1) 针对 ETL 涉及的源数据库中的数据表, 根据数据质量分析的结果, 建立数据抽取前的清洗函数。该清洗函数可由调度控制程序在数据抽取前进行统一调度, 也可分散到各个抽取函数中调度。

(2) 针对 ETL 涉及的源数据库中的数据表, 根据代码数据差异分析的结果, 对需要转换的代码数据值进行转换。如果数据长度无变化或变化不大, 考虑对源数据表中引用的代码在抽取前进行转换。抽取前转换需要建立代码转换函数。代码转换函数由调度控制程序在数据抽取前进行统一调度。

(3) 对新旧代码编码规则差异较大的代码, 考虑在抽取过程中进行转换。根据代码数据差异分析的结果, 调整所有涉及该代码数据的抽取函数。

4. 数据迁移后的校验

在数据迁移完成后, 需要对迁移后的数据进行校验。数据迁移后的校验是对迁移质量的检查, 同时数据校验的结果也是判断新系统能否正式启用的重要依据。可以通过以下两种方式对迁移后的数据进行校验:

(1) 对迁移后的数据进行质量分析: 可以通过数据质量检查工具, 或编写有针对性的检查程序进行。对迁移后数据的校验有别于迁移前历史数据的质量分析, 主要是检查指标的不同。迁移后数据校验的指标主要包括完整性检查、一致性检查、总分平衡检查、记录条数检查和特殊样本数据的检查。

(2) 新旧系统查询数据对比检查: 通过新旧系统各自的查询工具, 对相同指标的数据进行查询, 并比较最终的查询结果。先将新系统的数据恢复到现有系统迁移前一天的状态, 然后将最后一天发生在现有系统上的业务全部补充到新系统, 检查有无异常, 并和现有系统比较最终产生的结果。

15.8 现有系统演进

随着信息系统的运行和业务的发展, 对现有系统进行演进, 如对现有系统进行扩展升级等, 使其适应当前的应用情况, 就成为必然的, 也是经济的选择。而如果企业有多个应用系统, 可通过对这些系统进行集成, 使数据能在这些系统中共享。有关企业应用集成的知识, 在之前章节已有详细讨论, 此处不再赘述, 本节仅讨论系统的扩展问题, 以及扩展与集成的区别。

1. 系统扩展

系统的可扩展性是指将新的功能添加到系统中的能力。可扩展性可以分为动态可扩展性和静态可扩展性。动态可扩展性是指在系统运行的过程中, 能够添加新的功能, 而不会影响系统的其他部分; 静态可扩展性是指在添加新的功能时, 系统必须停止运行, 当新功能添加之后, 系统再重新启动。提高系统可扩展性的方法是在系统架构中减少构件之间的耦合。显然, 如果在系统规划和设计时充分考虑了可扩展性的因素, 在系统架构上进行了预留, 则同类型业务的扩展就相对容易些。

一般地, 当客户提出需求变更或增加新的功能时, 通常采用的方法首先就是对现有系统进行扩展, 以满足这种变化。扩展一般包括延伸型扩展和新建型扩展, 前者需要在理解扩展点附近的架构及代码的基础上, 以原有方式进行功能扩充; 后者则可能会完全另起炉灶, 在适应系统整体架构的前提下, 增加全新的功能。

通过在基本软件基础上,引入第三方软件包并进行二次开发的方式,可以迅速对系统功能进行扩展。例如,引入具有手写签批功能的控件,可以立即得到支持领导手工批阅公文的功能扩展。但这种引入也是双刃剑,需要在引入前进行充分的研究和分析,确保其能满足目前的扩展需求外,还具有适度前瞻的特性。同时,要求引入详细的设计文档甚至是源码,以保证对引入包的可控性,避免过度依赖第三方技术支持,降低实施风险。

2. 扩展与集成的比较

系统扩展和集成分别属于深度维护和广度维护活动,和开发过程类似,通常也需要经过需求分析、设计、编码、测试和实施的各环节来共同保证系统扩展和集成活动的最终成效。因此,需要分析人员、设计人员、编码人员、测试人员和实施人员参与其中,同时,也需要过程管理人员进行项目管理。此外,系统集成还特别需要组织的高层人员参与协调与沟通。

系统扩展的重点在设计阶段,为达到平滑扩展,需要仔细研究扩展点附近的软件环境,要避免因扩展引起原有系统的不稳定或功能失效,要进行细致的回归测试;系统集成的重点在分析阶段,为达到无缝集成,需要仔细分析业务,尤其是业务关联点,避免过度耦合、深度介入,增加集成复杂度。另外,还需要组织的高层领导强势协调,强调全局观念,互相配合,方能顺利进行集成。

在系统测试方面,系统扩展和集成的测试要进行全面的回归,不能“改头测头,改脚测脚”,系统集成尤其要重视接口的测试和全流程的测试。